第六章 同步

1. **Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.**

In single-processor systems, there is only one thread can run at a time. and no parallelism. It can make a busy waiting. and even deadlock happens.

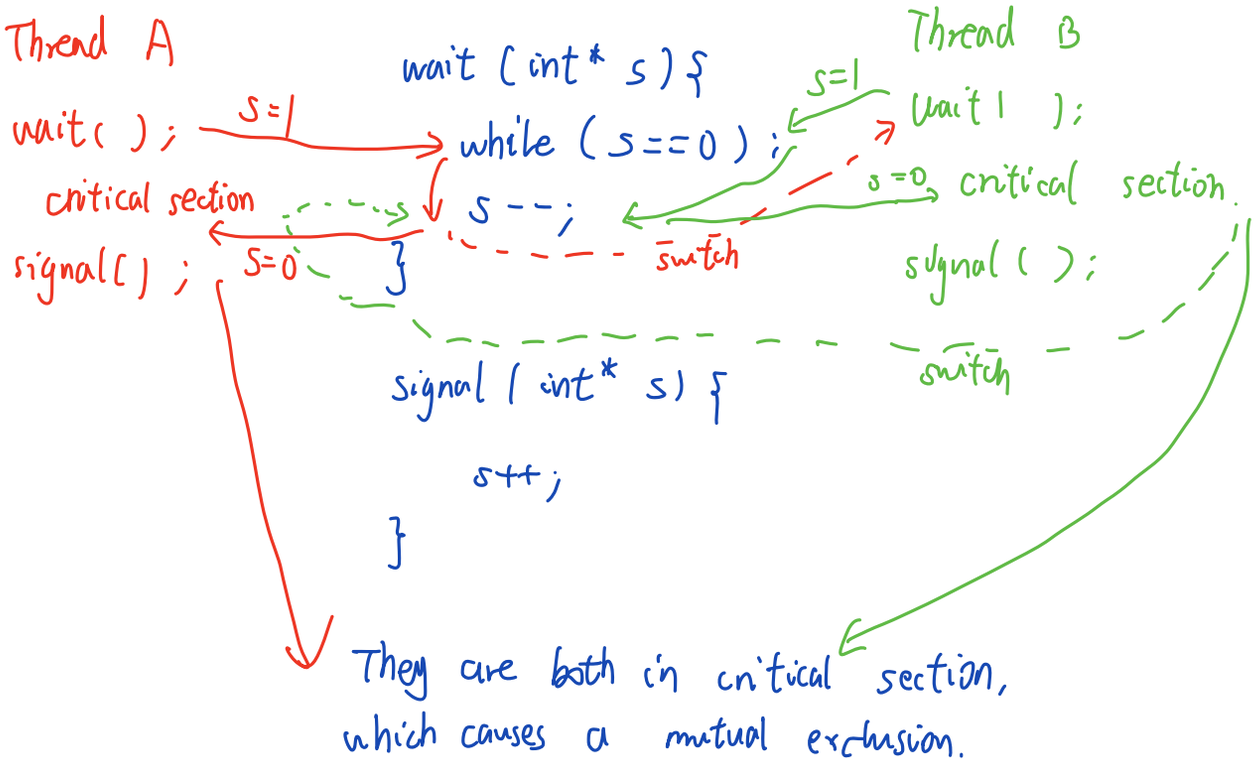
However, in multiprocessor systems, when a thread holds the lock, another thread who wants the lock just spins. waiting it to release the lock. It avoids context switching, which spends more time. and is good for short critical sections.

2. **Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.**

define: sema s=1;

```
wait (sema* s){
    while (s==0);
    s--;
}
```

```
signal (sema* s){
    s++;
}
```



3. **The Sleeping-Barber Problem. A barbershop consists of a waiting room with n chairs and abarber room with one barber chair. If there are no customers to be served, the barber goes to sleep. If a customer enters the barbershop and all chairs are occupied, then the customer leaves the shop. If the barber is busy but chairs are available, the customer sits in one of the free chairs. If the barber is asleep, the customer wakes up the barber. Write a program to coordinate the barber and the customers.**

```
int chairs;  int cus = 0;
Semaphore* barber = 0;
Semaphore* customer = 0;
Semaphore* mutex = 1;

Barber ( ) {
    wait (customers);
    wait (mutex);
    cus--;
    signal (barber);
    signal (mutex);
    cut hair ...
}

Customer ( ) {
    wait (mutex);
    if (customers < chairs) {
        signal (customers);
        cus++;
        signal (mutex);
        wait (barber);
        cut hair ...
    } else {
        signal (mutex);
        leave ...
    }
```

4. 有一个系统，定义 P、V 操作如下：

P(s):

s:=s-1;

if s<0 then

将本进程插入相应队列末尾等待；

V(s):

s:=s+1;

if s<=0 then

从相应等待队列队尾唤醒一个进程，将其插入就绪队列；

问题：

（1）这样定义 P、V 操作是否有问题？

（2）用这样的 P、V 操作实现 N 个进程竞争使用某一共享变量的互斥机制。

(1) 应从队首唤醒进程，而非队尾，这样会造成 starvation

(2)
```
semaphore mutex = 1;
Process() {
    P(mutex);
    critical section
    V(mutex);
}
```

5. P、V 操作实现第二类读者写者问题：写者优先。 Why not use a monitor?

具体条件如下：

多个读者可以同时进行读；

写者必须互斥（只允许一个写者写，也不能读者写者同时进行）；

写者优先于读者(一旦有写者，则后续读者必须等待，唤醒时优先考虑写者)。

Semaphore: rmutex = 1, wmutex = 1, readTry = 1, resource = 1

int: readers = 0, writers = 0

```
Reader() {
    P(readTry);
    P(rmutex);
    readcount++;
    if(readcount == 1)
        P(resource);
    V(rmutex);
    V(readTry);
```

```
Writer() {
    P(wmutex);
    writecount++;
    if(writecount == 1)
        P(readTry);
    V(wmutex);
    P(resource);
    Writing...
```

```
Reading ...
P(rmutex);
readcount--;
if(readcount==0)
    V(resource);
V(rmutex);
}
```

```
V(resource);
P(wmutex);
writecount--;
if(writecount==0)
  V(readTry);
V(wmutex);
}
```

6. 把学生和监考老师都看作进程，学生有 N 人，教师 1 人。考场门口每次只能进出一个人，进考场原则是先来先进。当 N 个学生都进入考场后，教师才能发卷子。学生交卷后可以离开考场。教师要等收上来全部卷子并封装卷子后才能离开考场。

问题:

（1）问共需设置几个进程？

（2）试用 P、V 操作解决上述问题中的同步和互斥关系。

(1) N+1个

(2) Semaphore: entry = 1, teacher_entry = 0, teacher_leave = 0
start = 0, mutex = 1

int: student = 0, paper = 0

```
Student() {
    P(entry);
    student ++;
    if(student == N)
        V(teacher_entry);
    V(entry);
    P(start);
    Get the paper ...
    V(start);
    Doing the paper ...
    P(mutex);
    paper ++;
    if(paper == N)
        V(teacher_leave);
    V(mutex);
    P(entry);
    Leave...
    V(entry);
}
```

```
Teacher() {
    P(teacher_entry);
    Teacher gives out the papers ...
    P(teacher_leave);
    Collect the papers ...
    P(entry);
    Teacher leaves the room ...
    V(entry);
}
```